1. Where would you rate yourself on (LLM, Deep Learning, AI, ML). A, B, C [A = can code independently; B = can code under supervision; C = have little or no understanding]

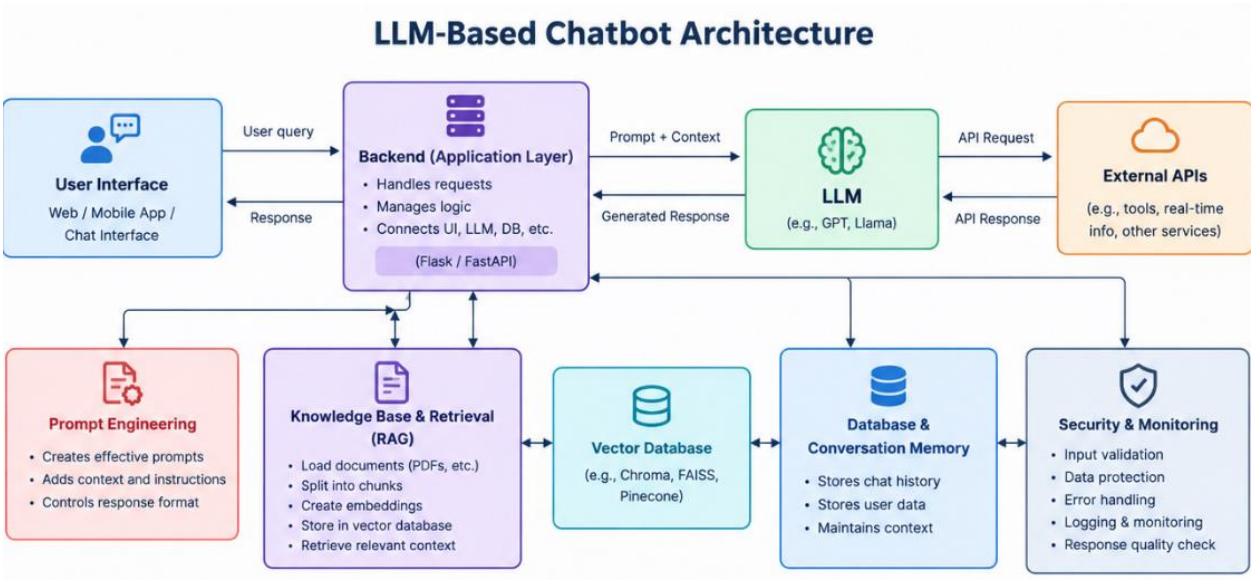
LLM -> B

Deep Learning -> C

AI -> A

ML -> A

2. What are the key architectural components to create a chatbot based on LLM? Please explain the approach on a high-level.



An LLM-based chatbot is an application that understands user questions and generates relevant responses using a Large Language Model. To build a chatbot, different components work together to receive user input, process it, generate a response, and display the result.

1. User Interface: This is where users interact with the chatbot by typing questions and viewing responses. It can be a web application, mobile application, or chat interface.

2. Backend: The backend receives the user's message, processes the request, and connects the user interface with the language model and other services. Python frameworks such as Flask or FastAPI can be used to build the backend.

3. Large Language Model (LLM): The LLM is the main component that understands the user's question and generates a response. Examples include GPT and Llama models.

4. Prompt Engineering: A prompt provides instructions and context to guide the model's response. A well-designed prompt helps the chatbot answer clearly and stay focused on the user's question.

5. Knowledge Base and Retrieval (RAG): If the chatbot needs to answer questions using specific documents or company information, we can use Retrieval-Augmented Generation (RAG). The documents are processed, divided into smaller chunks, and converted into embeddings. Relevant information is retrieved when a user asks a question and provided to the LLM as context.

6. Database and Conversation Memory: A database can store chat history, user information, and other required data. Conversation memory helps the chatbot maintain context across multiple messages.

7. API Integration: APIs allow the backend to communicate with an LLM provider or other external services. The backend sends the prompt to the model and receives its response.

8. Security and Monitoring: The chatbot should protect user data, validate inputs, handle errors, and monitor response quality and performance.

High-Level Approach

First, I would identify the purpose of the chatbot and decide whether it needs general knowledge or information from specific documents. Then, I would build a simple user interface and backend using Python. Next, I would integrate an LLM through an API and design suitable prompts. If the chatbot needs to answer questions from documents, I would add a RAG pipeline with a vector database to retrieve relevant information. Finally, I would test the chatbot for accuracy, response time, security, and reliability before deploying it.

3. Please explain vector databases. If you were to select a vector database for a hypothetical problem (you may define the problem) which one will you choose, and why?

A vector database is a type of database that stores and searches data in the form of vectors. Vectors are numerical representations of information such as text, images, or documents. These representations are called embeddings and are generated using embedding models.

Unlike traditional databases, which mainly search for exact matches, vector databases can find information based on meaning and similarity. For example, if a user searches for "How can I reset my password?", a vector database may retrieve a document containing "Steps to recover your account" because both have a similar meaning.

How Does a Vector Database Work?

First, the data is divided into smaller sections if needed. An embedding model converts each section into a numerical vector, which is stored in the vector database along with the original text and relevant metadata. When a user submits a query, the query is also converted into a vector. The database compares it with stored vectors and retrieves the most relevant results using similarity search. These results can then be passed to an LLM to generate an appropriate answer.

Hypothetical Problem

Suppose I want to build a PDF chatbot that allows users to upload documents and ask questions about their content. The chatbot should retrieve relevant information from the uploaded PDFs and provide accurate answers.

Which Vector Database Would I Choose and Why?

For this project, I would choose **Chroma** as the vector database.

My reasons are:

1. **Easy to use:** Chroma is beginner-friendly and can be integrated with Python applications.
2. **Suitable for RAG applications:** It can store document embeddings and retrieve relevant text for a chatbot.
3. **Metadata filtering:** It allows documents to be filtered using metadata, such as document names or categories.
4. **Local development:** It is convenient for building and testing projects locally without needing to manage a separate cloud database service.
5. **Integration:** It works well with common Python-based AI and RAG workflows.

Another Example: E-commerce Product Recommendation System

Suppose I want to build an e-commerce application that recommends products based on a customer's interests. For example, if a customer searches for "comfortable shoes for running," the system should find products that match the meaning of the search, even if their descriptions use different words.

For this problem, I would choose **Pinecone** because it is a managed cloud vector database that supports similarity search and can scale as the number of products and users grows. It can store product embeddings and metadata, such as product categories and prices, to help retrieve relevant products. This reduces the need to manage the vector database infrastructure myself.